

Perceptrón Multicapa – Reporte del Ejercicio 01

Objetivo de la actividad

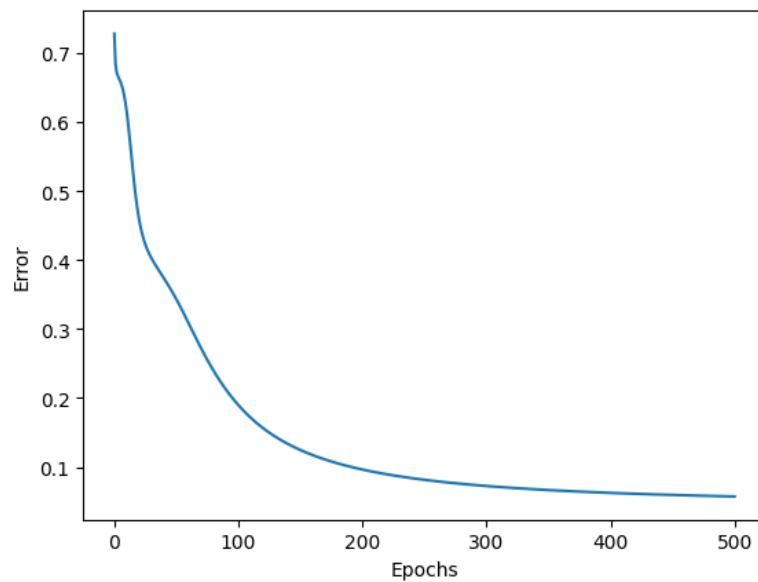
*Correr ambas notebooks en **Google Colab** con la arquitectura original, **agregar dos capas** a cada red, volver a entrenar y **comparar** qué cambia (curva de error/pérdida, velocidad, calidad de la clasificación).*

Notebooks con nuevas capas

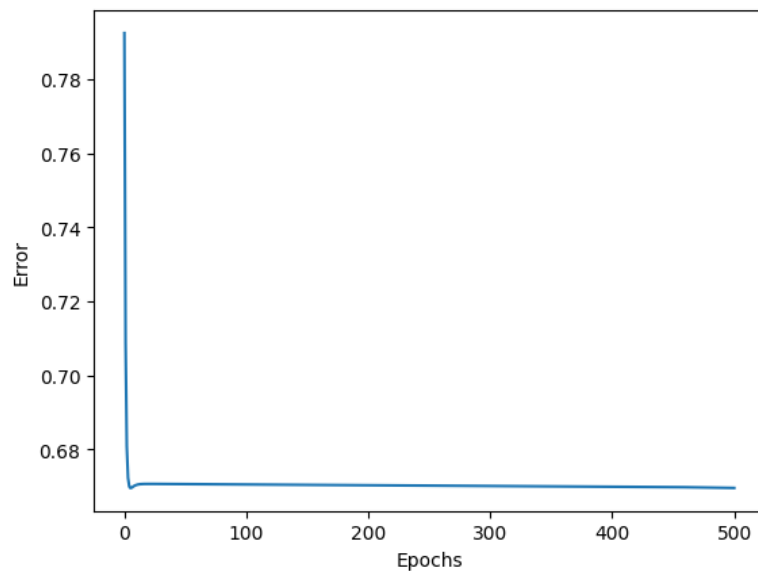
Agregué las nuevas notebooks dentro de la carpeta original, añadiendo el prefijo AlanPeraza_NombreArchivo.ipynb para que sea más sencillo de identificar. Debajo de la última celda original, hay una nueva sección con mi nombre.

Gráficas de Pérdida

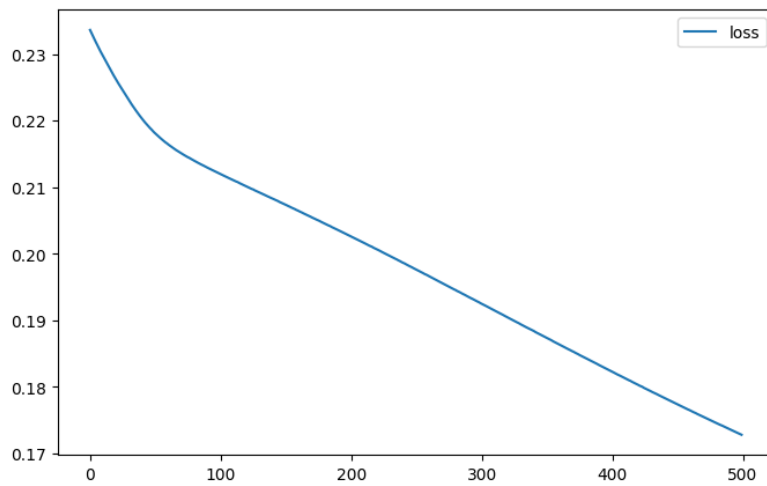
Multilayer Perceptron - Original



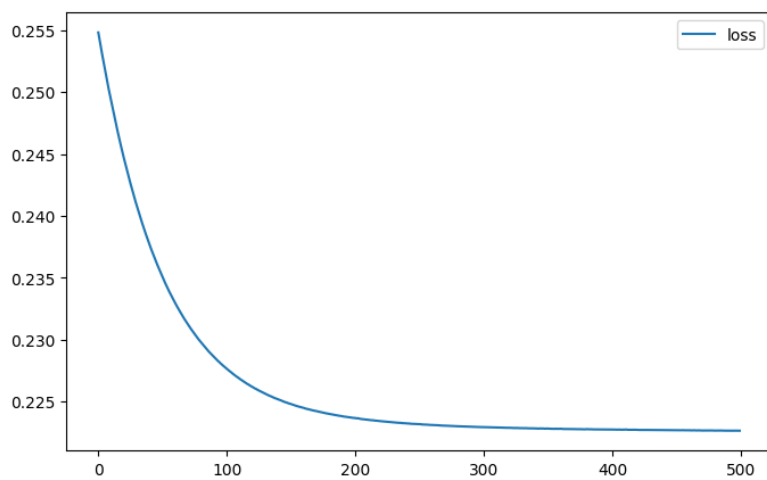
Multilayer Perceptron - Editado



Keras Multilayer Perceptron – Original



Keras Multilayer Perceptron - Editado



Reporte de resultados obtenidos

Un breve reporte (media página a una página) que responda:

- *¿Bajar más el error al añadir dos capas, o se estancó / empeoró? ¿Igual en NumPy y en Keras?*

Al aumentar la profundidad de la red de 1 a 3 capas ocultas, el error no sólo no disminuyó, sino que el aprendizaje se estancó. Esto sucedió en ambos perceptrones, tanto en el manual como el creado con Keras.

En las arquitecturas originales del multilayer perceptron (tanto Numpy como Keras) el error logró reducirse de manera considerable durante las 500 épocas. Pero cuando se añadieron más capas ocultas, la red no pudo reducir el error. De hecho, la gráfica muestra que no hubo cambios notorios en este caso. Las curvas de pérdida se estancaron.

- *¿Las curvas de la notebook 01 y de Keras se parecen con la misma topología? Si no, ¿qué diferencias de implementación podrían explicarlo (orden de los datos, inicialización, vectorización, etc.)?*

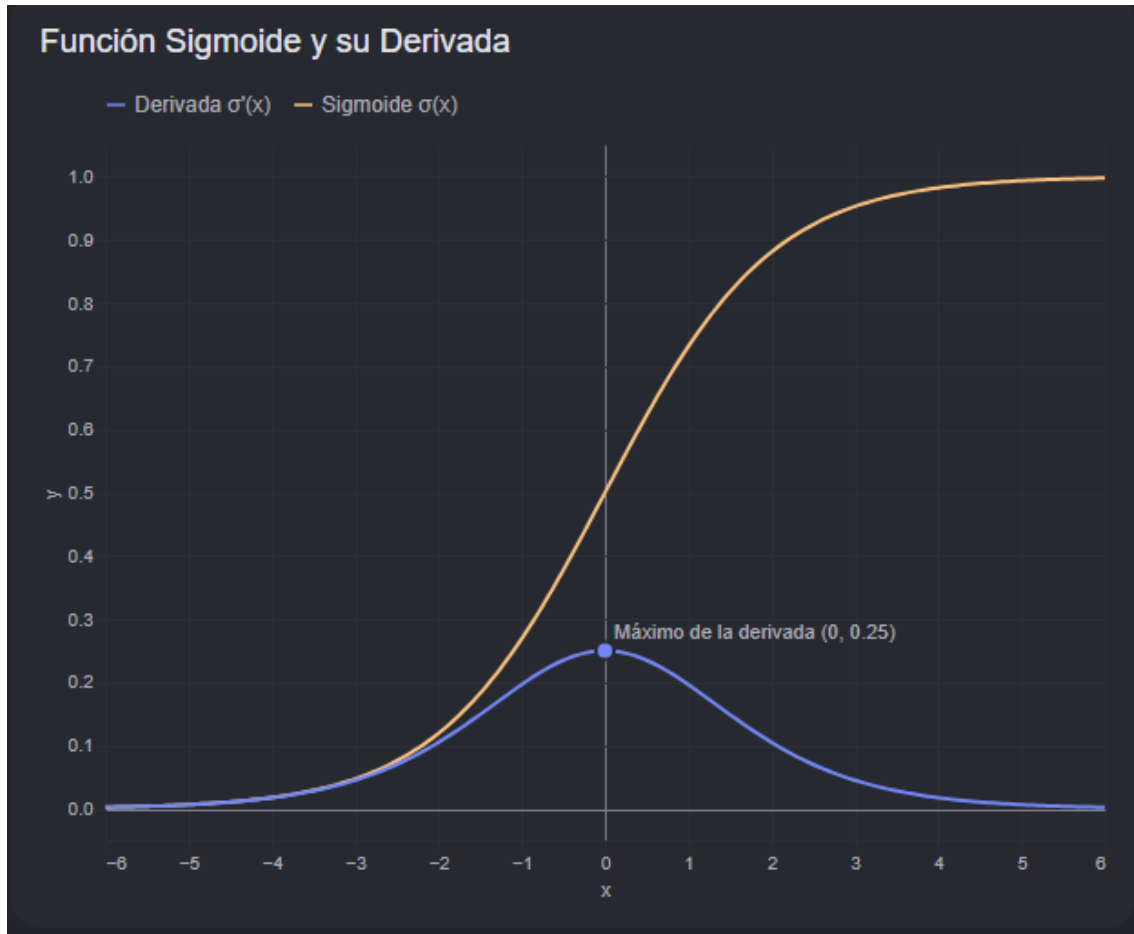
En los modelos con arquitectura original (sin añadir capas extra), las curvas no se parecen visualmente, pero ambas muestran una tendencia de disminución del error. La diferencia es mucho más evidente en los modelos con capas añadidas. Si bien ambas gráficas muestran un estancamiento en la disminución del error, la gráfica de error de Keras es mucho más suave si se compara con la de Numpy.

Las diferencias que podrían explicar esta discrepancia, a mi parecer, podrían ser las siguientes:

1. Orden de los datos: En el caso de Numpy, el dataset X se navega de forma secuencial – primero Iris, luego Versicolor, luego Virginica. Al no haber aleatoriedad en la lectura de los datos, la red podría tener un sesgo con la clase que se está procesando en cada set de muestras. Esto no sucede con Keras, que aplica un barajado aleatorio (shuffle) (referencia: [Método fit de la clase model](#)).
2. Vectorización: Numpy y Keras usan distintos tipos de flotantes. Numpy usa float64, mientras que Keras usa float32. Esto podría explicar las diferencias en aquellas operaciones que corren de manera recurrente.

- Con sigmoides apiladas y MSE, ¿tiene sentido que una red **más profunda** no aprenda mejor en Iris? Relaciónalo con lo que viste en las gráficas.

Podría tener sentido si se toma en cuenta el desvanecimiento del gradiente. La derivada de la función sigmoide es cuando mucho 0.25 (ocurre cuando $x = 0$).



Si tomamos en cuenta 4 capas, ese 0.25 se elevaría a la cuarta potencia, que sería aproximadamente .0039. Entonces, cada vez que se actualizan los pesos, son prácticamente nulos.

Evidencias de ejecución del código

Adjuntos capturas correspondientes a las Notebooks:

